RAPPORT

MINI PROJET

LYBRINTHIA



REALISE PAR:

Ben rabia Arij

Khelifi nourhen

Laribi Mayssa



Contexe de projet

- "**Lybrinthia**" est un jeu de lettres où le joueur explore une grille pour former des mots valides en se déplaçant entre les cases selon des règles spécifiques. Le score dépend de la longueur, de la complexité des mots et de l'optimisation du chemin. Ce mini-projet d'algorithmique vise à modéliser la grille sous forme de graphe, utiliser des algorithmes de parcours (BFS, DFS) pour l'exploration, et appliquer des méthodes de recherche du plus court chemin (Dijkstra) pour optimiser les déplacements, validant ainsi les compétences en manipulation de graphes et en gestion de structures de données.

Implémentation

Structure du code

- **lybrinthia.cpp** :Fichier principal (Main).
- **Test.cpp** :Interface finale du jeu.
- **Menus** (MainMenu, DifficultyMenu, OptionsMenu) : Gèrent l'affichage et les options.
- **Constants.hpp** : Définit les constantes globales.
- **Ressources** (doc.txt, fond_ecran.jpeg) :Contenu additionnel.

Langage de programmation

- **Performances** : C++ offre des performances optimales avec une exécution compilée.
- **Mémoire** : C++ permet un contrôle direct sur l'allocation mémoire.
- **Optimisation** : C++ exploite mieux les ressources matérielles.
- **Bibliothèques** : C++ est compatible avec des moteurs de jeux puissants.

Fonctionnalités supplémentaires

- **Trois niveaux de difficultés:**

Facile: un seul thème sans chrono.
Moyen: plusieurs thèmes sans chrono.
Difficile: plusieurs thèmes avec un chrono limité à 2 min.

- **Interfaces graphique.**
- **Le score :** est calculé sur deux parties. la premier sur le longueur des mots trouvés et la deuxième sur le longueur de chemin trouvé.
- **Affichage des mots trouvés au fur et à mesure du parcourt .**

Règle du jeu

Le jeu est divisé en deux parties:
Partie 1: chercher les mots valides.
Partie 2: passer à trouver le chemin le plus court possible pour arriver au point d'arrivé.
et le score sera attribuer selon le longueur des mots trouvés ainsi que du chemin trouvé.
le joueur est face à deux choix soit il va joueur les deux parties en même temps soit séparés pour augmenter sa chance d'obtenir un score maximale.

Recherche du plus court chemin

Dans notre labyrinthe, le graphe a des poids uniformes, donc la distance équivaut au nombre d'arêtes traversées. Ainsi, bien que nous utilisions Dijkstra avec une file de priorité, il se comporte comme un BFS, trouvant toujours le chemin avec le moins d'étapes.

Validation des mots

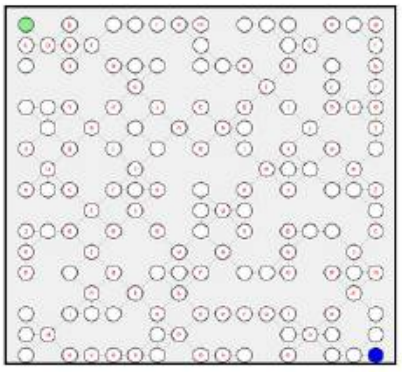
- **getWordFromPath() :** construire un mot à partir d'un chemin de joueur.
- **findWords():** Recherche tous les mots valides accessibles depuis une position donnée.
- **isValidWord():** Vérifie si un mot est présent dans la liste des mots valides.

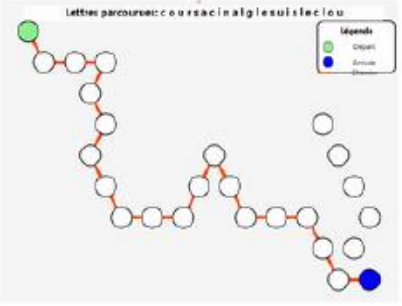
Fonctionnalités principales

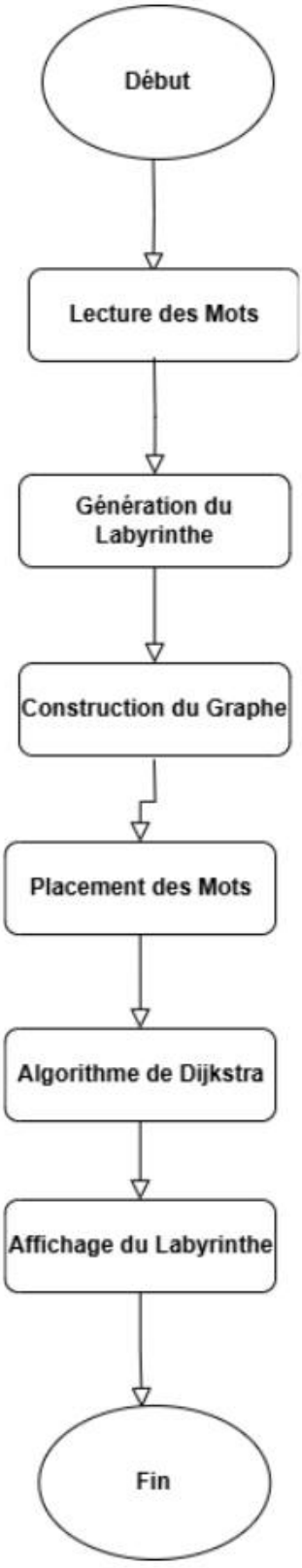












Fonction **readWordsFromFile** : Cette fonction lit un fichier texte contenant des mots (par exemple doc.txt) et les ajoute dans un vecteur de chaînes de caractères.

Fonction **generateMaze** : Cette fonction génère un labyrinthe en utilisant une approche récursive. Elle commence à une cellule de départ et explore les directions possibles, en ajoutant de nouveaux chemins (cellules de type PATH) et en supprimant les murs (cellules de type WALL).

Fonction **buildGraph** : Transforme le labyrinthe en un graphe représenté par une matrice d'adjacence. Chaque cellule du labyrinthe qui est un chemin (PATH) est représentée par un nœud dans le graphe. Chaque nœud est relié aux voisins accessibles par des arêtes de poids 1.

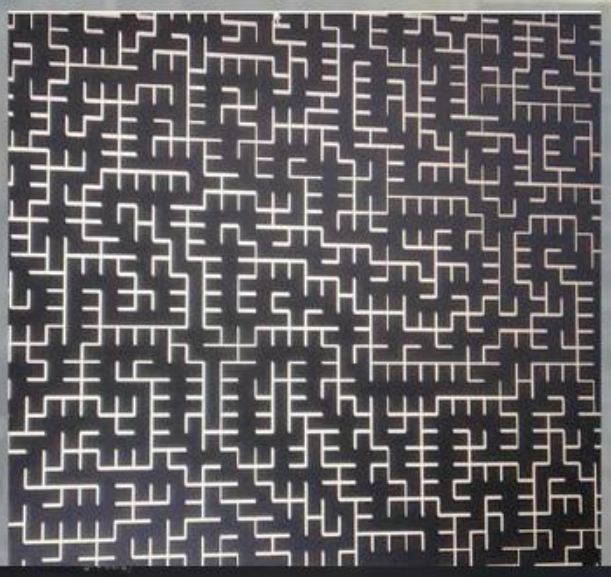
Fonction **placeWords** : insère les mots dans le labyrinthe en suivant un chemin aléatoire. Si un emplacement libre est trouvé, le mot est placé ; sinon, il est abandonné après plusieurs tentatives. Les cases vides restantes sont remplies de lettres aléatoires.

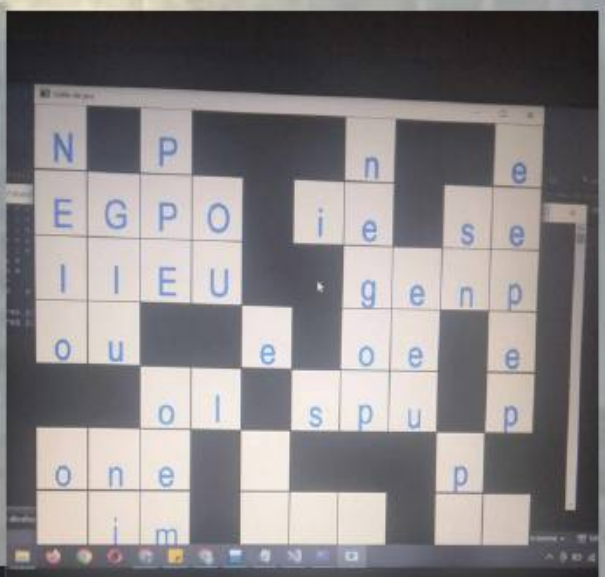
Fonction **Dijkstra** : Utilise l'algorithme de Dijkstra pour trouver le chemin le plus court dans un graphe via un min-heap. Les distances minimales sont mises à jour, puis le chemin est retracé depuis l'arrivée jusqu'au départ.

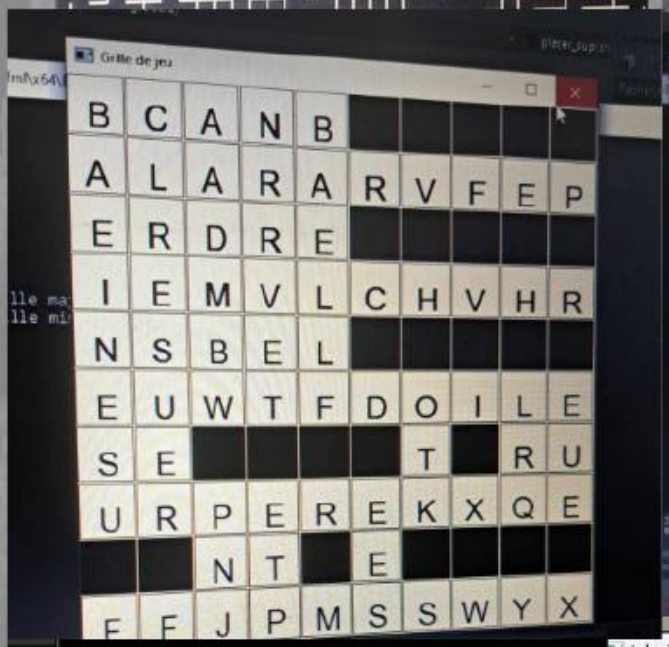
Fonction **displayMaze** : Utilise SFML pour afficher le labyrinthe. Les cellules PATH sont blanches, les WALL sont noires. Les lettres et le chemin le plus court (marqué par *) sont aussi affichés.

Etapes de réalisation de projet

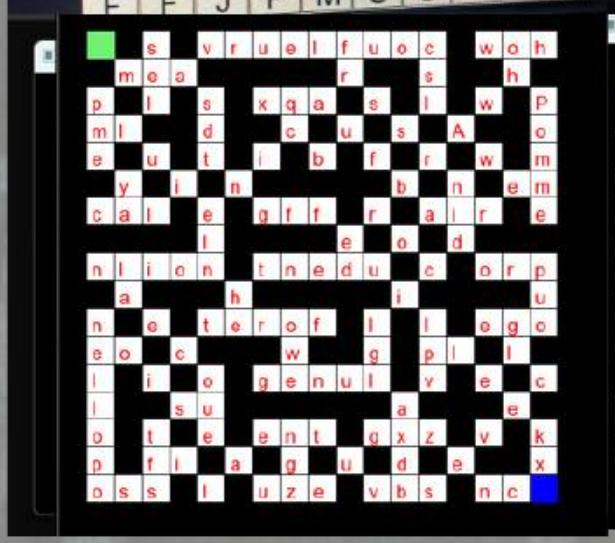


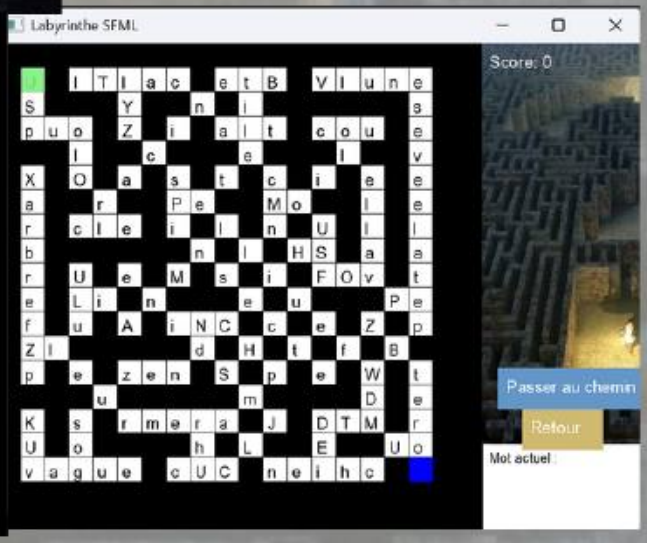












Etapes de réalisation de projet

